SoFlow

Referenz zum Theme-System

# Anleitung für eigene Themes


Kurzfassung

Eigene Themes ersetzen den Fold-Bereich (den oberen Teil des Haupt-Dashboards) durch dein eigenes HTML-Layout. Du lieferst HTML für die Struktur und CSS für die Optik; die App füllt deine Elemente automatisch alle 500 ms mit Live-Scooter-Daten, ganz ohne JavaScript auf deiner Seite. Dieses Dokument ist die vollständige Referenz: das Bundle-Format, jeder Live-Daten-Hook, die Live-Zustandssignale, die Tacho-Technik, die CSS-Variablen, das Sicherheitsmodell und ein vollständiges Beispiel.



Autor Laufbursche42


Guide-Version 1.0

Repository github.com/Laufbursche42/sf-lb-edition

# Inhalt

**1 Was ein Theme ist und was nicht** **2**

**2 Das Bundle-Format** **2**

**3 Theme-Scope** **2**
3.1 Full-Scope-DOM-Gerüst . . . 3

**4 theme.html - Struktur und Hooks** **3**
4.1 Live-Wert-Hooks - data-lb . . . 3
4.2 Zustands-Hooks - data-lb-state . . . 4
4.3 Aktions-Hooks - data-lb-action . . . 4
4.4 Kachel-Registrierung - data-tile-id . . . 4
4.5 Verfügbare Datenschlüssel . . . 4

**5 Live-Zustandssignale auf `<html>`** **5**
5.1 Drossel-/Entsperrzustand . . . 5
5.2 Anzeigeeinheit . . . 6

**6 Kreisförmiger Tachometer - vollständige Technik** **6**
6.1 Geometrie . . . 6
6.2 Den Füllbogen mit --cur-speed animieren . . . 6
6.3 Verwaltete Skalen-Marken (optional) - data-lb-gauge . . . 7
6.4 Einheiten-Umschalter . . . 8
6.5 Inline-SVG-Icons . . . 8
6.6 Vollständiger Minimal-Gauge . . . 8

**7 theme.css - Layout und Variablen** **9**
7.1 Von der App bereitgestellte CSS-Variablen . . . 9

**8 theme.json - Metadaten** **12**

**9 CSS-Variablen-Overrides - cssVars** **12**
9.1 Label-Lokalisierung . . . 13

**10 Sicherheitsmodell - kein JavaScript** **14**

**11 Import-Anleitung** **14**
11.1 Schritt 1 - ZIP erstellen . . . 14
11.2 Schritt 2 - Importieren . . . 15

**12 Portabilität zwischen Laufbursche-Apps** **15**

**13 Übersetzungen und Sprache** **15**
13.1 Vorhandene App-Strings verwenden . . . 15
13.2 Eigene Übersetzungen hinzufügen . . . 16

**14 Vollständiges Beispiel - Orbit (Full-Scope-Lehr-Theme)** **16**
14.1 theme.json . . . 17
14.2 theme.html . . . 17

14.3 theme.css . . . . . 19

**15 Tipps und Einschränkungen** **21**

# 1 Was ein Theme ist und was nicht

Ein Custom-Theme **ersetzt den HTML-Inhalt von `#fold-inner`** - den Teil des Hauptbildschirms, der oberhalb des Scroll-Bereichs liegt. Layout, Typografie und Farben steuerst du mit eigenem HTML und CSS. Die App treibt alles Übrige weiter: die Scroll-Kacheln darunter, die Schnellaktions-Button-Reihe, das Aufnahme-Banner und alle Sub-Pages.

**Ein Theme ist keine Web-App.** Es kann kein JavaScript ausführen, keine externen Ressourcen laden und die App-Logik nicht ändern. Es ist nur Struktur und Style - die Daten kommen von der App.

---

# 2 Das Bundle-Format

Ein Theme ist eine **ZIP-Datei**, die genau drei Dateien im Wurzelverzeichnis des Archivs enthält:

| Datei | Pflicht | Zweck |
|---|---|---|
| `theme.html` | **Ja** | HTML, das ins Dashboard injiziert wird (siehe Theme-Scope unten) |
| `theme.css` | **Ja** | CSS, das für dieses Theme geladen wird (auf dein HTML beschränkt) |
| `theme.json` | Nein | Metadaten: Anzeigename, Version, Scope, optionale CSS-Variablen-Deklarationen |

Der ZIP-Dateiname wird zum Fallback-Namen beim Import, wenn `theme.json` kein `name`-Feld hat.

**Größenlimit:** 2 MB pro Datei.

---

# 3 Theme-Scope

Das `scope`-Feld in `theme.json` steuert, wie viel vom Dashboard dein Theme ersetzt.

| `scope`-Wert | Injektionsziel | Verhalten |
|---|---|---|
| `"fold"` (Standard, oder Feld weglassen) | `#fold-inner` | Ersetzt nur den Hero-Abschnitt oben am Bildschirm. Da |
| `"full"` | `#scroller` | Ersetzt das **gesamte Dashboard** - sowohl den Hero-Ab |

**fold-Scope** eignet sich am besten für leichte Reskins: Du gestaltest den oberen Hero-Bereich neu und überlässt die Datenanzeige darunter den Standard-Kacheln.

**full-Scope** ist für komplette Dashboard-Redesigns gedacht: Dein `theme.html` muss sowohl den Hero-Abschnitt als auch alle gewünschten Datenkacheln enthalten. Alle `data-lb-`, `data-lb-state-` und `data-lb-action`-Hooks funktionieren in beiden Scopes identisch.

Beispiel-`theme.json` für ein Full-Scope-Theme:

```
{
  "name": "Mein Dashboard",
```

```
  "scope": "full",
  "version": "1.0",
  "author": "du"
}
```

Ein Full-Scope-`theme.html` sollte seinen Inhalt als Abschnitte strukturieren, die von oben nach unten fließen, jeder mit `data-tile-id` registriert, damit der Kachel-Manager sie ein-/ausblenden und neu anordnen kann.

### 3.1 Full-Scope-DOM-Gerüst

Es gibt keinen erforderlichen äußeren Wrapper. Abschnitte fließen in Dokumentreihenfolge:

```
<!-- Kein äußerer Wrapper nötig - Abschnitte fließen in Dokumentreihenfolge. -->

<!-- Abschnitt 1: Hero-/Gauge-Bereich (oben am Bildschirm) -->
<div data-tile-id="my-hero" data-tile-label="Tacho">
  <!-- Hero-Inhalt, SVG-Gauge usw. -->
</div>

<!-- Abschnitt 2: Datenkachel-Raster -->
<div data-tile-id="my-grid" data-tile-label="Daten">
  <!-- Datenzellen mit data-lb-Schlüsseln -->
</div>

<!-- Abschnitt 3: Schnellaktions-Button-Reihe (unten) -->
<div data-tile-id="my-buttons" data-tile-label="Buttons">
  <!-- Aktionsbuttons mit data-lb-action -->
</div>
```

Regeln für Full-Scope:

- Jedes direkte Kind (oder jeder Nachfahre) kann `data-tile-id` tragen.
- Der Kachel-Manager listet jeden registrierten Abschnitt über sein `data-tile-label`.
- Abschnitte ohne `data-tile-id` sind immer sichtbar und nicht verwaltbar.
- Die eingebauten Scroll-Kacheln (`#scroller`-Inhalt) sind für die Dauer des Themes verschwunden; deine Abschnitte stellen die gesamte Datenanzeige bereit.

---

## 4 theme.html - Struktur und Hooks

`theme.html` ist ein reines HTML-Fragment - **keine `<html>`-, `<head>`- oder `<body>`-Tags**, nur der Inhalt, den du in `#fold-inner` (Fold-Scope, Standard) oder `#scroller` (Full-Scope - siehe Theme-Scope) haben möchtest. Die App bereinigt es vor dem Speichern (siehe Sicherheitsmodell).

### 4.1 Live-Wert-Hooks - data-lb

Füge `data-lb="key"` zu einem beliebigen Element hinzu. Die App setzt den `textContent` dieses Elements alle 500 ms auf den aktuellen Rollerwert:

```
<div class="my-speed" data-lb="speed"></div>
<div class="my-soc"   data-lb="soc"></div>
```

Werte sind Zahlen (oder `true`/`false` für Booleans). Formatiere sie mit CSS (`content` auf `::after` für Einheiten, oder einfach Text neben dem Wert). Wenn der Roller getrennt ist, bleibt der letzte bekannte Wert bis zur nächsten Verbindung erhalten.

## 4.2 Zustands-Hooks - data-lb-state

Füge `data-lb-state="key:classname"` hinzu, um eine CSS-Klasse je nach Wahrheitswert eines Datenwerts zu schalten:

```
<div class="speed-box" data-lb-state="park:is-parked">
```

Wenn `data.park` `true` ist, wird die Klasse `is-parked` hinzugefügt; bei `false` wird sie entfernt. Nutze das für visuelle Zustandsänderungen (anderer Hintergrund beim Parken, Blinken bei Sperre usw.).

## 4.3 Aktions-Hooks - data-lb-action

Eine eingebaute Aktion wird unterstützt:

| Wert | Verhalten |
|---|---|
| `speed-unlock` | Dreifach-Tipp auf dieses Element löst die eKFV-Entsperrsequenz aus (wie die Standard-Tacho-Ka |

```
<div class="my-speed" data-lb="speed" data-lb-action="speed-unlock"></div>
```

## 4.4 Kachel-Registrierung - data-tile-id

Umschließe Abschnitte deines Themes mit einem Element mit `data-tile-id` und `data-tile-label`, um sie beim Kachel-Manager zu registrieren. Registrierte Kacheln erscheinen in der Kachelliste des Theme-Managers, können ein- oder ausgeblendet und vom Nutzer per Drag & Drop neu angeordnet werden:

```
<div data-tile-id="my-speed-block" data-tile-label="Tacho">
  <!-- Tacho-Inhalt -->
</div>
<div data-tile-id="my-motor-block" data-tile-label="Motoren">
  <!-- Motor-Inhalt -->
</div>
```

Ohne `data-tile-id` ist ein Abschnitt immer sichtbar und nicht verwaltbar.

## 4.5 Verfügbare Datenschlüssel

Alle Schlüssel kommen aus der BLE-Telemetrie des Rollers, alle 500 ms geparst. Rohe Zahlen sind `0`, wenn der Roller getrennt ist; `park` ist `false`.

| Schlüssel | Typ | Einheit | Beschreibung |
|---|---|---|---|
| `speed` | number | km/h oder mph | Aktuelle Geschwindigkeit |
| `avgSpeed` | number | km/h oder mph | Durchschnittsgeschwindigkeit (diese Fahrt) |
| `maxSpeed` | number | km/h oder mph | Höchstgeschwindigkeit (diese Fahrt) |
| `soc` | number | % | Ladezustand |
| `soh` | number | % | Gesundheitszustand Akku |
| `volt` | number | V | Paketspannung |

| Schlüssel | Typ | Einheit | Beschreibung |
|---|---|---|---|
| `poleVolt` | number | V | Polspannung |
| `current` | number | A | Paketstrom (null wenn nicht gemeldet) |
| `gear` | number | - | Aktiver Gang (1-3) |
| `rearCurrent` | number | A | Hintermotor-Strom |
| `frontCurrent` | number | A | Vordermotor-Strom |
| `rTemp` | number | °C | Hintermotor-Temperatur |
| `fTemp` | number | °C | Vordermotor-Temperatur |
| `power` | number | kW | Gesamtleistung |
| `singleMile` | number | km oder mi | Tageskilometer |
| `totalMile` | number | km oder mi | Gesamtkilometer |
| `fb` | number | - | Rückspeisung/Regen-Wert |
| `rssi` | number | dBm | Bluetooth-Signalstärke |
| `park` | boolean | - | `true` wenn Roller geparkt |
| `vcuUnlock` | number | - | `0` = gesperrt, `1` = entsperrt, `-1` = unbekannt |

**Einheitenhinweis:** `speed`, `avgSpeed`, `maxSpeed`, `singleMile` und `totalMile` folgen automatisch der km/mph-Einstellung der App - die Zahl, die du erhältst, ist bereits in der vom Nutzer gewählten Einheit.

**Werteformat:** alle Zahlen sind rohe JavaScript-Floats, z.B. `speed` = `25.3`, `power` = `0.24`. Ohne JavaScript gibt es keine Formatierungskontrolle. Wenn du eine Ganzzahlanzeige brauchst: CSS kann Zahlen nicht runden; gestalte dein Layout so, dass ein Wert mit einer Nachkommastelle passt (breiterer Container, oder akzeptiere einfach die Genauigkeit, die der Roller meldet).

---

# 5 Live-Zustandssignale auf `<html>`

Die App setzt Klassen und CSS-Variablen direkt auf dem `<html>`-Element, damit script-freies Theme-CSS auf Live-Zustände reagieren kann, ohne ein `data-lb-state`-Attribut auf einem bestimmten Element zu brauchen.

## 5.1 Drossel-/Entsperrzustand

| Signal | Typ | Werte | Wann gesetzt |
|---|---|---|---|
| `lb-unlocked` | Klasse auf `<html>` | vorhanden / abwesend | Vorhanden, wenn der Roller entdrosse |
| `--throttle-unlocked` | CSS-Variable auf `:root` | `1` / `0` | `1` = entsperrt, `0` = gesperrt; nützlich |

Wird bei jedem BLE-Tick (500 ms) aktualisiert. Standard beim Seitenstart: abwesend / `0`.

**CSS-Verwendung:**

```
/* Skalenring zeigt bei Entsperrung den vollen Bereich 0-100 */
.gauge-scale { --scale-max: 40; }
:root.lb-unlocked .gauge-scale { --scale-max: 100; }

/* Oder mit der numerischen Variable */
```

```
.gauge-arc {
  stroke-dasharray: calc(var(--throttle-unlocked) * 220 + 80);
}
```

### 5.2 Anzeigeeinheit

| Signal | Typ | Werte | Wann gesetzt |
|---|---|---|---|
| `lb-mph` | Klasse auf `<html>` | vorhanden / abwesend | Vorhanden, wenn der Nutzer mph gewählt hat; abwesend f |

Wird beim Start und bei jedem Einheitenwechsel in den Einstellungen gesetzt.

**CSS-Verwendung:**

```
.speed-unit::after { content: 'km/h'; }
:root.lb-mph .speed-unit::after { content: 'mph'; }
```

---

## 6 Kreisförmiger Tachometer - vollständige Technik

Ein kreisförmiger Tachometer ist das häufigste Element in Full-Scope-Themes. Dieser Abschnitt liefert dir das exakte SVG-Markup und die CSS-Berechnungen, um einen von Grund auf zu bauen.

### 6.1 Geometrie

Der Tacho besteht aus zwei übereinander liegenden `<circle>`-SVG-Elementen:

- **Track-Ring** - voller 270-Grad-Hintergrundbogen in Grau.
- **Füllbogen** - kürzerer Bogen, der mit `--cur-speed` wächst.

Die Technik heißt `stroke-dasharray` / `stroke-dashoffset`:

```
stroke-dasharray="DRAW GAP"
```

- `DRAW` = Länge des sichtbaren Segments (SVG-User-Units).
- `GAP` = Lücke, die den Rest des Kreises füllt.

Für Radius 58 in einem 160×160 viewBox:

- Umfang = 2 ×  × 58  364 px
- 270° = 75% von 364  **273 px** (DRAW)
- Lücke = 364 - 273 = **91 px**

Also: `stroke-dasharray="273 91"` zeichnet einen 270°-Bogen.

Standardmäßig beginnt der Bogen bei 3 Uhr (SVG-Null-Winkel). 135° um den Mittelpunkt drehen, um bei 7 Uhr zu beginnen:

```
transform="rotate(135 80 80)"
```

### 6.2 Den Füllbogen mit --cur-speed animieren

Steuere die sichtbare Länge des Füllbogens mit `stroke-dashoffset`:

- `dashoffset = 0` → volle 273 px sichtbar (Maximalgeschwindigkeit).

- `dashoffset = 273` → Bogen vollständig verborgen (0 km/h).
- Formel: `offset = 273 - speed × 273 / maxSpeed`

Definiere in CSS `--speed-max` auf dem Element und referenziere `--cur-speed` von `:root`:

```
.gauge-arc {
  --speed-max: 25;   /* eKFV gesperrt: 25 km/h */
  stroke-dashoffset: calc(273 - var(--cur-speed) * 273 / var(--speed-max));
  transition: stroke-dashoffset 0.4s ease-out;
}

/* Skala erweitern, wenn der Roller entsperrt ist (lb-unlocked auf <html>). */
:root.lb-unlocked .gauge-arc { --speed-max: 100; }
```

Kein JavaScript nötig - `--cur-speed` wird von der App alle 500 ms aktualisiert, und CSS berechnet automatisch neu.

### 6.3 Verwaltete Skalen-Marken (optional) - data-lb-gauge

Bei den eingebauten Themes Carbon und Lambo kann der Nutzer die Zahlen-Skala des Tachos wählen - **Klar**, **Fein** oder **Rund** - unter **Einstellungen -> Theme-Manager -> „Tacho-Skala"**. Die App zeichnet die Tick-Marken und Zahlen-Labels (gleichmäßig verteilt, damit dichte niedrige Werte wie 20 / 22 / 25 lesbar bleiben) und steuert den Zeiger so, dass er exakt auf der gewählten Zahl steht.

Dein eigenes Full-Scope-Theme kann das nutzen, statt die Skala selbst zu zeichnen: Füge deinem Tacho-`<svg>` ein `data-lb-gauge`-Attribut hinzu und lass zwei leere Gruppen für die App zum Füllen.

```
<svg class="my-gauge"
     data-lb-gauge='{"prefix":"my","cx":100,"cy":100,"labelR":61,"tickR0":84,"tickR1":94,"arcR":
     viewBox="0 0 200 200" aria-hidden="true">
  <!-- dein statischer Track-Ring, Zeiger usw. -->
  <g class="my-scale scale-locked"></g>
  <g class="my-scale scale-unlocked"></g>
</svg>
```

| Feld | Bedeutung |
|---|---|
| `prefix` | Klassen-Präfix für die von der App eingefügten Elemente (`my-tick`, `my-lbl`, `my-redzone-hi`) - |
| `cx` / `cy` | Tacho-Mittelpunkt, in viewBox-Einheiten |
| `labelR` | Radius der Zahlen-Labels |
| `tickR0` / `tickR1` | innerer / äußerer Radius jeder Tick-Linie |
| `arcR` | Radius des Rotzonen-Bogens |
| `arcPct` | Prozent des Vollkreises, den der 270-Grad-Bogen einnimmt (`75`) |

Die App füllt `.my-scale.scale-locked` mit den gesperrten Marken und `.scale-unlocked` mit den offenen (km/h in `.lbl-kmh`, mph in `.lbl-mph`, umgeschaltet per `lb-mph`), fügt einen `.my-redzone-hi`-Bogen hinzu und setzt jeden Tick `--cn-f` (0-1, der stückweise Zeiger-Anteil) auf dem `<svg>`. Steuere deinen Zeiger und Füllbogen damit, z. B. `transform: rotate(calc(135deg + var(--cn-f) * 270deg))`. Lässt du `data-lb-gauge` weg, bleibt die voll manuelle reine-CSS-Skala von oben.

### 6.4 Einheiten-Umschalter

Zwei überlappende `<text>`-Elemente per `opacity` umschalten. `display:none` ist bei SVG-Text in Android WebView unzuverlässig.

```
<text class="g-unit g-unit-kmh" x="80" y="100" text-anchor="middle">km/h</text>
<text class="g-unit g-unit-mph" x="80" y="100" text-anchor="middle">mph</text>
```

```
.g-unit-kmh { opacity: 1; }
.g-unit-mph { opacity: 0; }
:root.lb-mph .g-unit-kmh { opacity: 0; }
:root.lb-mph .g-unit-mph { opacity: 1; }
```

### 6.5 Inline-SVG-Icons

Keine `url()`- oder `http://`-Quellen erlaubt. SVG direkt einbetten:

```
<svg class="my-icon" viewBox="0 0 24 24" fill="currentColor" aria-hidden="true">
  <path d="M7 2v11h3v9l7-12h-4l4-8z"/>   <!-- Blitz -->
</svg>
```

`fill="currentColor"` erbt die CSS-`color`-Eigenschaft, Einfärben ist also einfach `color: var(--value)`.

### 6.6 Vollständiger Minimal-Gauge

```
<svg class="g-gauge" viewBox="0 0 160 160" aria-hidden="true">
  <!-- Track-Ring: statischer 270°-Hintergrundbogen.
       rotate(135 80 80) setzt den Start auf 7 Uhr. -->
  <circle class="g-track"
    cx="80" cy="80" r="58" fill="none"
    stroke-dasharray="273 91"
    transform="rotate(135 80 80)" />
  <!-- Füllbogen: stroke-dashoffset per CSS aus --cur-speed gesteuert. -->
  <circle class="g-arc"
    cx="80" cy="80" r="58" fill="none"
    stroke-dasharray="273 91"
    transform="rotate(135 80 80)" />
  <text class="g-val" x="80" y="80" text-anchor="middle" dominant-baseline="middle"
        data-lb="speed">0</text>
  <text class="g-unit g-unit-kmh" x="80" y="100" text-anchor="middle">km/h</text>
  <text class="g-unit g-unit-mph" x="80" y="100" text-anchor="middle">mph</text>
</svg>
```

```
.g-gauge { width: 100%; max-width: 200px; overflow: visible; }

.g-track {
  stroke: var(--border);
  stroke-width: 10;
  stroke-linecap: round;
}
```

```css
.g-arc {
  --speed-max: 25;
  stroke: var(--value);
  stroke-width: 10;
  stroke-linecap: round;
  stroke-dashoffset: calc(273 - var(--cur-speed) * 273 / var(--speed-max));
  transition: stroke-dashoffset 0.4s ease-out;
}
:root.lb-unlocked .g-arc { --speed-max: 100; }

.g-val       { font-size: 38px; font-weight: 700; fill: var(--text); font-family: inherit; }
.g-unit      { font-size: 12px; fill: var(--label); font-family: inherit; }
.g-unit-kmh  { opacity: 1; }
.g-unit-mph  { opacity: 0; }
:root.lb-mph .g-unit-kmh { opacity: 0; }
:root.lb-mph .g-unit-mph { opacity: 1; }
```

---

# 7 theme.css - Layout und Variablen

`theme.css` wird als gescopetes Stylesheet nach dem App-eigenen CSS geladen. Schreibe Selektoren, die deine eigenen Klassennamen ansprechen, oder Element-IDs, die du in `theme.html` definiert hast.

Du musst Farben oder Schriften **nicht** von Grund auf festlegen. Nutze die CSS-Variablen der App, und dein Theme passt sich automatisch an Dunkelmodus, Hellmodus und jeden pro-Theme-Stil an, den der Nutzer konfiguriert hat.

## 7.1 Von der App bereitgestellte CSS-Variablen

Diese Variablen werden von der App immer auf `:root` definiert:

### Runtime-Farb-Tokens - diese in `theme.css` verwenden

Dies sind die modusaufgelösten Tokens. Sie wechseln automatisch zwischen den Dunkel-/Hellwerten, wenn der Nutzer den Modus ändert. Immer diese in Selektoren und `var()`-Referenzen in deinem CSS verwenden.

| Variable | Beschreibung |
|---|---|
| `--card` | Kachel-/Card-Hintergrund |
| `--bg` | Seitenhintergrund |
| `--text` | Primärtext |
| `--label` | Sekundär-/gedämpfter Text |
| `--border` | Rahmen-/Trennlinienfarbe |
| `--value` | Akzent-/Highlight-Farbe (vom Nutzer einstellbar) |
| `--ble` | BLE-Geschwindigkeitsanzeige-Farbe |
| `--gps` | GPS-Geschwindigkeitsanzeige-Farbe |
| `--good` | Positiv-/OK-Zustandsfarbe |
| `--warn` | Warnfarbe |
| `--danger` | Fehler-/Kritischfarbe |
| `--unlock` | ”Entsperrt / entdrosselt”-Anzeigefarbe |

| Variable | Beschreibung |
|---|---|
| `--btnfill` | Button-Füllfarbe |

### Per-Modus-Quell-Tokens - diese in `cssVars` deklarieren, um die Palette anzupassen

Dies sind die per-Modus-Werte, die die Runtime-Tokens oben speisen. Deklariere sie in `cssVars` in `theme.json`, um die eigene Palette deines Themes festzulegen. Der Style-Editor zeigt sie nach Modus gruppiert.

| Variable | Standard (dunkel) | Standard (hell) | Beschreibung |
|---|---|---|---|
| `--dark-card` | `#111420` | | Kachel-Hintergrund im Dunkelmodus |
| `--dark-bg` | `#0A0C10` | | Seitenhintergrund im Dunkelmodus |
| `--dark-border` | `#1C2035` | | Rahmenfarbe im Dunkelmodus |
| `--dark-text` | `#E4EDF6` | | Primärtext im Dunkelmodus |
| `--dark-label` | `#C6D2E2` | | Sekundärtext im Dunkelmodus |
| `--dark-lum` | `90` | | Helligkeit 0-100 im Dunkelmodus |
| `--light-card` | | `#ffffff` | Kachel-Hintergrund im Hellmodus |
| `--light-bg` | | `#e8edf5` | Seitenhintergrund im Hellmodus |
| `--light-border` | | `#CDD5E4` | Rahmenfarbe im Hellmodus |
| `--light-text` | | `#0A0D18` | Primärtext im Hellmodus |
| `--light-label` | | `#0A0D18` | Sekundärtext im Hellmodus |
| `--light-lum` | | `100` | Helligkeit 0-100 im Hellmodus |

### Akzentfarben-Standards

Diese in `cssVars` deklarieren, um die Standard-Akzentpalette des Themes festzulegen. Nutzer können sie im Style-Editor überschreiben.

| Variable | Standard | Beschreibung |
|---|---|---|
| `--value` | `#BCEB39` | Primäre Akzent-/Wertfarbe |
| `--ble` | `#B8F020` | BLE-Geschwindigkeitsanzeige |
| `--gps` | `#3DB8FF` | GPS-Geschwindigkeitsanzeige |
| `--good` | `#1EE89A` | Positiv-Zustandsfarbe |
| `--btnfill` | `#B8F020` | Button-Füllung |
| `--warn` | `#FFAA18` | Warnfarbe |
| `--danger` | `#FF5050` | Fehler-/Kritischfarbe |
| `--unlock` | `#FF5050` | Entsperrt-Anzeigefarbe |

### Typografie - feste Skala, nicht editierbar

| Variable | Standard | Beschreibung |
|---|---|---|
| `--fs-2xs` | 10 px | Kleinster Text (Badges, Tags) |
| `--fs-xs` | 11 px | Sehr kleiner Text |
| `--fs-sm` | 12 px | Kleiner Text |
| `--fs-md` | 13 px | Mittlerer Text |

| Variable | Standard | Beschreibung |
|---|---|---|
| `--fs-base` | 14 px | Grundtext |
| `--fs-xl` | 16 px | Großer Text |
| `--fw-semibold` | 600 | Halbfett-Gewicht |
| `--fw-bold` | 700 | Fett-Gewicht |

**Schriftgrößen der Kachelelemente** (vom Nutzer über den Style-Editor einstellbar; in `cssVars` deklarieren, um Theme-Standards festzulegen)

| Variable | Standard | Beschreibung |
|---|---|---|
| `--sz-speed-val` | 64 px | Tacho-Zahl |
| `--sz-speed-lbl` | 14 px | Tacho-Beschriftung |
| `--sz-hero-val` | 38 px | Hero-Kachel-Wert |
| `--sz-hero-lbl` | 13 px | Hero-Kachel-Beschriftung |
| `--sz-m1-top` | 12 px | Motor-Kachel-1-Titel |
| `--sz-m1-mid` | 32 px | Motor-Kachel-1-Wert |
| `--sz-m1-bot` | 12 px | Motor-Kachel-1-Einheit |
| `--sz-m2-top` | 12 px | Motor-Kachel-2-Titel |
| `--sz-m2-mid` | 32 px | Motor-Kachel-2-Wert |
| `--sz-m2-bot` | 12 px | Motor-Kachel-2-Einheit |

**Kachel-Mindesthöhen** (vom Nutzer einstellbar; `0` = natürliche Inhaltshöhe)

| Variable | Standard | Beschreibung |
|---|---|---|
| `--tile-h-speed` | 0 px | Mindesthöhe Tacho-Kachel |
| `--tile-h-hero` | 0 px | Mindesthöhe Hero-Kachel |
| `--tile-h-motors` | 0 px | Mindesthöhe Motor-Kachel |

**Transparenz** (vom Nutzer über den Style-Editor einstellbar; für jedes Theme immer gesetzt)

Der Style-Editor zeigt "Transparenz / Durchsichtigkeit"-Schieberegler. **Reglerskala: 0 % = vollständig opak (Standard), 100 % = vollständig transparent.** Die injizierte CSS-Variable ist immer die **Deckkraft** (1.0 = opak, 0.0 = unsichtbar); die App rechnet intern mit `opacity = 1 - slider / 100` um.

| Variable | CSS-Wert | Standard | Beschreibung |
|---|---|---|---|
| `--tile-alpha` | 0.0-1.0 | `1.0` | Deckkraft der Kacheln / Cards. 1.0 = vollständig opak, 0.0 = unsichtba |
| `--btn-alpha` | 0.0-1.0 | `1.0` | Deckkraft der Buttons. 1.0 = vollständig opak, 0.0 = unsichtbar. |
| `--gauge-alpha` | 0.0-1.0 | `1.0` | Deckkraft des Gauge-Füllbogens. 1.0 = vollständig opak, 0.0 = unsichtb |

**Verwendung in `theme.css`:**

```
/* Einfach: Deckkraft auf das gesamte Element anwenden (betrifft auch Text) */
[data-tile-id] { opacity: var(--tile-alpha); }
.lb-btn        { opacity: var(--btn-alpha);  }
```

```
.gauge-arc     { opacity: var(--gauge-alpha); }

/* Nur Hintergrund-/Fill-Transparenz (CSS Color Level 4 Relative-Color-Syntax,
   Chromium 111+ / Android WebView 111+): */
[data-tile-id] { background-color: rgb(from var(--card)    r g b / var(--tile-alpha)); }
.lb-btn        { background-color: rgb(from var(--btnfill) r g b / var(--btn-alpha)); }
.gauge-arc     { fill:             rgb(from var(--value)   r g b / var(--gauge-alpha)); }
```

> **Achtung:** `rgb(from color-mix(...) r g b / alpha)` funktioniert auf Android WebView **nicht** - ein `color-mix()`-Ursprung verwirft den Alpha-Kanal lautlos. Als Ursprung immer eine **flache CSS-Variable** verwenden (z.B. `var(--card)`, `var(--value)`, `var(--btnfill)`), niemals ein `color-mix()`-Ergebnis. Das gilt für alle drei Alpha-Variablen.

**Live-Telemetrie**

| Variable | Einheit | Beschreibung |
|---|---|---|
| `--cur-speed` | km/h (immer) | Aktuelle Geschwindigkeit, rohe km/h unabhängig von der Anzeigeeinheit-Einstellu |

**Form**

| Variable | Standard | Beschreibung |
|---|---|---|
| `--radius` | 12 px | Standard-Eckenradius |

---

## 8 theme.json - Metadaten

Optional. Falls vorhanden, muss es gültiges JSON sein:

```
{
  "name": "Mein Theme",
  "version": "1.0"
}
```

| Feld | Beschreibung |
|---|---|
| `name` | Anzeigename, der in der Theme-Liste der App angezeigt wird. Fällt auf den ZIP-Dateinamen zurück, w |
| `version` | Freier Versionsstring, nur informativ. |
| `scope` | `"fold"` (Standard) oder `"full"`. Wie viel vom Dashboard das Theme ersetzt. Siehe Theme-Scope. |
| `cssVars` | Objekt mit CSS-Variablen-Overrides. Diese werden zur Standard-Palette des Themes und im Style-Edit |
| `strings` | Übersetzungstabelle für eigene i18n-Schlüssel. Siehe Übersetzungen und Sprache. |

---

## 9 CSS-Variablen-Overrides - cssVars

Deklariere `cssVars` in `theme.json`, um die Standard-Farb-/Größenpalette deines Themes festzulegen. Die Werte laden in den Style-Editor als Ausgangspunkt, bevor der Nutzer etwas überschreibt.

Merge-Reihenfolge: **vom Nutzer gespeichertes Override > Theme-cssVars-Standard > App-interner Standard.**

Zwei Formate werden akzeptiert:

**Flacher String** (minimal - nur Wert):

```
{
  "name": "Mein Theme",
  "cssVars": {
    "--value": "#F4B223",
    "--dark-card": "#1A1500",
    "--dark-bg": "#0D0F00"
  }
}
```

**Objekt-Format** (bevorzugt - im Style-Editor mit Labels und Abschnitten angezeigt):

```
{
  "name": "Mein Theme",
  "cssVars": {
    "--value":      { "default": "#F4B223", "label": "Accent",      "group": "accents" },
    "--dark-card":  { "default": "#1A1500", "label": "Kachel-HG",   "group": "dark"    },
    "--dark-bg":    { "default": "#0D0F00", "label": "Hintergrund", "group": "dark"    },
    "--light-card": { "default": "#FFF8E0", "label": "Kachel-HG",   "group": "light"   },
    "--light-bg":   { "default": "#F5EFD0", "label": "Hintergrund", "group": "light"   }
  }
}
```

Gültige `group`-Werte, die Abschnitten im Style-Editor entsprechen: `"dark"`, `"light"`, `"accents"`, `"fonts"`, `"tiles"`. Jede Variable, die du in `cssVars` auflistest, kann vom Nutzer im Style-Editor weiter überschrieben werden. Deklariere deine vollständige eigene Palette in `cssVars`, um sie für Nutzeranpassungen verfügbar zu machen.

## 9.1 Label-Lokalisierung

Das `label`-Feld in einem `cssVars`-Eintrag akzeptiert drei Formate:

**1. i18n-Schlüssel (String)** - wird über den eingebauten Übersetzungskatalog der App aufgelöst. Für jede Variable, deren Name einem App-Standard-Token entspricht (z.B. `--warn`, `--dark-lum`). Die App liefert dafür bereits EN + DE:

```
"--warn": { "default": "#FFB020", "label": "style.var.warn", "group": "accents" }
```

**2. Zweisprachiges Objekt** - gib `en` und `de` selbst an. Für theme-spezifische Namen, die nicht im App-Katalog existieren:

```
"--value": { "default": "#22CCFF", "label": { "en": "Accent (arc)", "de": "Akzent (Bogen)" }, "g
```

**3. Einfacher String** - wird in beiden Sprachen unverändert angezeigt. Akzeptabel für rein dekorative oder Markennamen, die keine Übersetzung haben:

```
"--value": { "default": "#22CCFF", "label": "Arc accent", "group": "accents" }
```

Die Fallback-Kette ist: `{en, de}`-Objekt → Schlüssel-Lookup → einfacher String. Ein fehlender `de`-Schlüssel fällt auf `en` zurück.

---

# 10 Sicherheitsmodell - kein JavaScript

**JavaScript läuft in Themes nicht.** Das wird in zwei Ebenen erzwungen:

**Ebene 1 - Import-Sanitizer (Java, `ThemeManager.sanitizeHtml`).** Wird beim ZIP-Import angewandt, bevor das HTML gespeichert wird. Folgendes wird bedingungslos entfernt:

- `<script>`-Tags und ihr Inhalt
- `<iframe>`-Tags und ihr Inhalt (können beliebige Dokumente einbetten)
- `<base>`-Tags (würden die Auflösung relativer URLs ändern)
- Alle Inline-Event-Handler (`onclick=`, `onerror=`, `onload=` usw.)
- `javascript:`-URIs in den Attributen `href`, `src`, `action` und `formaction`
- `data:`-URIs in `src` und `href` (können HTML- oder JS-Payloads tragen)
- Externe `http://`- und `https://`-URLs in `src` und `href`

**Ebene 2 - Load-Sanitizer.** Derselbe Sanitizer läuft erneut, wenn das Theme von der Festplatte gelesen und in die Seite injiziert wird, sodass gespeicherte Themes, die älter als ein Sanitizer-Update sind, bei jedem Laden erneut geprüft werden.

**Warum Skripte auch ohne den Sanitizer nicht funktionieren würden:** `<script>`-Tags, die per `innerHTML` injiziert werden, werden von modernen Browsern und Android WebView nicht ausgeführt. Der Sanitizer existiert als Defense-in-Depth, nicht als einzige Schutzlinie.

**Was du statt JavaScript tun kannst:**

- `data-lb-state` für bedingte CSS-Klassen verwenden.
- CSS `@keyframes`, Transitions und Custom Properties für Animationen verwenden.
- `data-lb-action` für den eingebauten Speed-Unlock-Tipp verwenden.

---

# 11 Import-Anleitung

## 11.1 Schritt 1 - ZIP erstellen

Alle drei Dateien müssen **im Wurzelverzeichnis des Archivs** liegen (nicht in einem Unterordner). Die App sucht `theme.html`, `theme.css` und `theme.json` unabhängig vom Pfad nach Dateinamen, ein Unterordner funktioniert also auch, aber die Wurzel ist sauberer.

**Windows:** alle drei Dateien im Explorer markieren, Rechtsklick → "In ZIP-Datei komprimieren" (Windows 11) oder "Senden an → ZIP-komprimierter Ordner".

**macOS / Linux-Terminal:**

```
zip mein-theme.zip theme.html theme.css theme.json
```

**Android (für schnelle Tests):** einen Datei-Manager mit ZIP-Unterstützung verwenden (z.B. MT Manager, ZArchiver).

### 11.2 Schritt 2 - Importieren

1. ZIP auf dein Handy übertragen (AirDrop, Kabel, Cloud-Laufwerk usw.).
2. In der App: **Einstellungen → Display → Theme-Manager → Theme importieren**.
3. ZIP im Datei-Browser auswählen.
4. Das Theme erscheint in der Liste; auf **Aktivieren** tippen, um es anzuwenden.
5. Ein **Theme → Style**-Button öffnet den Style-Editor, in dem der Nutzer Farben, Helligkeit und Schriftgrößen zusätzlich zu deinem Theme überschreiben kann.

Um ein bereits importiertes Theme zu exportieren (zum Teilen oder Sichern): **Theme-Manager → (Theme antippen) → Exportieren**.

---

## 12 Portabilität zwischen Laufbursche-Apps

Das ZIP-Bundle-Format und die `data-lb`-Attributnamen sind eine **gemeinsame API aller Laufbursche-Apps**. Ein für eine App gebautes Theme kann in jede andere Laufbursche-App importiert werden, die Custom Themes unterstützt.

Wenn du ein Theme baust, das über mehrere Apps hinweg genutzt werden soll:

- Nur die `data-lb`-Schlüssel aus Verfügbare Datenschlüssel verwenden - das ist der stabile app-übergreifende Satz.
- Nur die CSS-Variablen aus Von der App bereitgestellte CSS-Variablen verwenden.
- Verlasse dich nicht auf feste IDs oder Klassen, die zum HTML einer einzelnen App gehören (wie `#speed-row` aus dem Standard-Layout).

Apps können über diese Liste hinaus zusätzliche app-spezifische Schlüssel oder CSS-Variablen bereitstellen; prüfe die Dokumentation jeder App.

---

## 13 Übersetzungen und Sprache

Die App läuft auf Deutsch oder Englisch. Beim Sprachwechsel führt die App den vollständigen i18n-Durchlauf über das gesamte Dokument erneut aus, einschließlich deines injizierten Theme-HTML.

### 13.1 Vorhandene App-Strings verwenden

Füge `data-i18n="key"` zu einem beliebigen Element hinzu, und die App ersetzt dessen Textinhalt zur Laufzeit durch den übersetzten String. Der Fallback-Text zwischen den Tags wird beim Entwickeln im Browser angezeigt:

```
<span data-i18n="dash.hero.gear.label">Gear</span>
```

Du kannst nur Schlüssel referenzieren, die bereits in der eingebauten i18n-Tabelle der App existieren. Nützliche eingebaute Schlüssel für Theme-Elemente:

| Schlüssel | DE | EN |
|---|---|---|
| `dash.hero.gear.label` | Gang | Gear |
| `dash.hero.battery.unit` | Batterie % | Battery % |
| `dash.motors.section` | Motoren | Motors |
| `dash.motor.rear.title` | HINTEN | REAR |

| Schlüssel | DE | EN |
|---|---|---|
| `dash.motor.front.title` | VORNE | FRONT |
| `dash.motor.power.title` | LEISTUNG | POWER |
| `dash.speed.vcu.unit` | km/h - VCU | km/h - VCU |
| `dash.speed.gps.unit` | km/h - GPS | km/h - GPS |

`data-i18n-aria`, `data-i18n-title` und `data-i18n-alt` funktionieren für die Attribute `aria-label`, `title` und `alt` genauso.

### 13.2 Eigene Übersetzungen hinzufügen

Deklariere einen `strings`-Block in `theme.json`. Die App merged diese beim Laden deines Themes in die globale i18n-Tabelle, sodass sie genau wie eingebaute Schlüssel funktionieren:

```
{
  "name": "Mein Theme",
  "version": "1.0",
  "strings": {
    "de": {
      "mtheme.speed.label": "Tempo",
      "mtheme.gear.label":  "Gang",
      "mtheme.unit.kmh":    "km/h"
    },
    "en": {
      "mtheme.speed.label": "Speed",
      "mtheme.gear.label":  "Gear",
      "mtheme.unit.kmh":    "km/h"
    }
  }
}
```

Dann in `theme.html`:

```
<span class="lbl" data-i18n="mtheme.speed.label">Speed</span>
<span class="val" data-lb="speed">-</span>
<span class="unit" data-i18n="mtheme.unit.kmh">km/h</span>
```

**Schlüsselbenennung:** prefixe deine Schlüssel mit etwas, das für dein Theme eindeutig ist (z.B. dem Theme-Namen), um Kollisionen mit eingebauten Schlüsseln oder anderen Themes zu vermeiden. Ein fehlender Schlüssel fällt auf den Fallback-Text zurück, den du zwischen die Tags geschrieben hast, sodass die Oberfläche lesbar bleibt, selbst wenn ein Schlüssel undefiniert ist.

**Nur eine Sprache:** wenn dein Theme nur auf ein Publikum zielt, codiere die Labels einfach direkt im HTML ohne `data-i18n`. Das ist völlig in Ordnung und hält das Bundle einfacher.

---

## 14 Vollständiges Beispiel - Orbit (Full-Scope-Lehr-Theme)

Orbit ist ein vollständiges, importierbares Theme, das jede Technik aus dieser Anleitung demonstriert: einen kreisförmigen SVG-Tachometer, `lb-unlocked`-Skalenerweiterung, `lb-mph`-Einheitenwechsel, al-

le drei Transparenz-Variablen verdrahtet, ein Datengitter, einen Button mit Inline-SVG-Icon und einen Park-Zustandsindikator. Speichere die drei Dateien unten, komprimiere sie zu `Orbit.zip` und importiere direkt.

### 14.1 theme.json

```
{
  "name": "Orbit",
  "version": "1.0",
  "scope": "full",
  "cssVars": {
    "--value":     "#BCEB39",
    "--btnfill":   "#BCEB39",
    "--dark-card": "#111420",
    "--dark-bg":   "#0A0C10"
  }
}
```

### 14.2 theme.html

```
<!-- ORBIT - scope:"full" ersetzt #scroller (gesamter Dashboard-Scroll-Bereich).
     Drei data-tile-id-Abschnitte lassen den Nutzer jeden Block ein-/ausblenden und neu anordnen
     Alle Klassennamen haben das Präfix ot-, um Konflikte mit App-Styles zu vermeiden. -->

<!-- HERO: Tachometer + SOC-Zeile.
     opacity: var(--tile-alpha) in theme.css verdrahtet den Kachel-Transparenz-Schieberegler. --
<div class="ot-hero" data-tile-id="ot-hero" data-tile-label="Tacho">

  <!-- Inline-SVG-Gauge - keine externen Bilder oder Schriften.
       viewBox "0 0 160 160"; CSS legt die gerenderte Größe fest.
       Umfang bei r=58: ~364 px. 270°-Bogen = 273 px; Lücke = 91 px.
       rotate(135 80 80): Start bei 7 Uhr statt 3 Uhr. -->
  <svg class="ot-gauge" viewBox="0 0 160 160" aria-hidden="true">

    <!-- Track-Ring: statischer grauer 270°-Hintergrundbogen. -->
    <circle class="ot-track"
      cx="80" cy="80" r="58" fill="none"
      stroke-dasharray="273 91"
      transform="rotate(135 80 80)" />

    <!-- Füllbogen: stroke-dashoffset per CSS aus --cur-speed berechnet.
         --gauge-alpha verdrahtet den Tacho-Transparenz-Schieberegler (siehe theme.css). -->
    <circle class="ot-arc"
      cx="80" cy="80" r="58" fill="none"
      stroke-dasharray="273 91"
      transform="rotate(135 80 80)" />

    <!-- data-lb="speed" verdrahtet den Live-km/h-Wert; App setzt textContent alle 500 ms. -->
    <text class="ot-speed-val" x="80" y="76" text-anchor="middle"
```

```
          data-lb="speed">-</text>

    <!-- Einheiten-Labels: zwei überlappende <text>-Elemente per CSS-opacity umgeschaltet.
         display:none ist bei SVG-Text in Android WebView unzuverlässig. -->
    <text class="ot-unit ot-unit-kmh" x="80" y="97" text-anchor="middle">km/h</text>
    <text class="ot-unit ot-unit-mph" x="80" y="97" text-anchor="middle">mph</text>

    <!-- Sperr-Badge: gleiche opacity-Umschalttechnik. -->
    <text class="ot-badge ot-badge-locked"   x="80" y="126" text-anchor="middle">GESPERRT</text>
    <text class="ot-badge ot-badge-unlocked" x="80" y="126" text-anchor="middle">ENTSPERRT</text
  </svg>

  <!-- SOC-Zeile. data-lb-state fügt ot-park-on hinzu, wenn der Roller geparkt ist. -->
  <div class="ot-soc-row">
    <span class="ot-soc-val" data-lb="soc">-</span><span class="ot-soc-unit">%</span>
    <span class="ot-park" data-lb-state="park:ot-park-on">P</span>
  </div>
</div>

<!-- DATENGITTER -->
<div class="ot-grid" data-tile-id="ot-grid" data-tile-label="Daten">
  <div class="ot-cell">
    <span class="ot-lbl">Volt</span>
    <span class="ot-val" data-lb="volt">-</span>
    <span class="ot-usm">V</span>
  </div>
  <div class="ot-cell">
    <span class="ot-lbl">Strom</span>
    <span class="ot-val" data-lb="current">-</span>
    <span class="ot-usm">A</span>
  </div>
  <div class="ot-cell">
    <span class="ot-lbl">Gang</span>
    <span class="ot-val" data-lb="gear">-</span>
  </div>
  <div class="ot-cell">
    <span class="ot-lbl">Trip</span>
    <span class="ot-val" data-lb="singleMile">-</span>
    <span class="ot-usm">km</span>
  </div>
</div>

<!-- BUTTON-REIHE.
     data-lb-action="speed-unlock" verdrahtet den eingebauten Dreifach-Tipp-Entsperrer.
     Das Icon nutzt fill="currentColor", um die CSS-color-Eigenschaft zu erben. -->
<div class="ot-buttons" data-tile-id="ot-buttons" data-tile-label="Buttons">
  <div class="ot-btn" data-lb-action="speed-unlock">
    <svg class="ot-btn-icon" viewBox="0 0 24 24" fill="currentColor" aria-hidden="true">
      <path d="M7 2v11h3v9l7-12h-4l4-8z"/>
```

```
    </svg>
    <span class="ot-btn-lbl">Entsperren</span>
  </div>
</div>
```

### 14.3 theme.css

```
/* === ORBIT  -  ot- Präfix hält alle Namen für dieses Theme eindeutig === */

/* --- HERO ------------------------------------------------------------
   opacity: var(--tile-alpha) verdrahtet den "Kachel-Transparenz"-Schieberegler.
   --tile-alpha ist ein 0-1-Float (1 = vollständig opak). */
.ot-hero {
  display: flex;
  flex-direction: column;
  align-items: center;
  padding: 16px 12px 8px;
  background: var(--card);
  border-radius: var(--radius);
  margin-bottom: 8px;
  opacity: var(--tile-alpha);
}

/* Gauge-SVG: füllt den Container bis 200 px.
   overflow:visible lässt den Strich außerhalb des viewBox-Rands zeichnen. */
.ot-gauge { width: 100%; max-width: 200px; overflow: visible; }

/* Track-Ring: statischer Hintergrundbogen. */
.ot-track {
  stroke: var(--border);
  stroke-width: 10;
  stroke-linecap: round;
}

/* Füllbogen: von --cur-speed gesteuert.
   --speed-max: 25 = eKFV-Limit; erweitert sich auf 110 bei lb-unlocked.
   stroke-dashoffset = 273 (Bogen verborgen) bei Tempo 0,
                     = 0   (voller Bogen) bei Tempo == --speed-max.
   --gauge-alpha verdrahtet den "Tacho-Transparenz"-Schieberegler. */
.ot-arc {
  --speed-max: 25;
  stroke: rgb(from var(--value) r g b / var(--gauge-alpha));
  stroke-width: 10;
  stroke-linecap: round;
  stroke-dashoffset: calc(273 - var(--cur-speed) * 273 / var(--speed-max));
  transition: stroke-dashoffset 0.4s ease-out;
}
:root.lb-unlocked .ot-arc { --speed-max: 100; }
```

```
.ot-speed-val { font-size: 38px; font-weight: var(--fw-bold); fill: var(--text); font-family: in

/* Einheiten-Labels: opacity-Umschalter - display:none ist bei SVG-Text
   in Android WebView unzuverlässig. */
.ot-unit       { font-size: 12px; fill: var(--label); font-family: inherit; }
.ot-unit-kmh   { opacity: 1; }
.ot-unit-mph   { opacity: 0; }
:root.lb-mph .ot-unit-kmh { opacity: 0; }
:root.lb-mph .ot-unit-mph { opacity: 1; }

/* Sperr-/Entsperr-Badge: gleiche opacity-Technik. */
.ot-badge            { font-size: 9px; letter-spacing: 0.08em; font-family: inherit; }
.ot-badge-locked     { fill: var(--warn);   opacity: 1; }
.ot-badge-unlocked   { fill: var(--value);  opacity: 0; }
:root.lb-unlocked .ot-badge-locked   { opacity: 0; }
:root.lb-unlocked .ot-badge-unlocked { opacity: 1; }

.ot-soc-row  { display: flex; align-items: baseline; gap: 4px; margin-top: 6px; }
.ot-soc-val  { font-size: 22px; font-weight: var(--fw-bold); color: var(--value); }
.ot-soc-unit { font-size: var(--fs-sm); color: var(--label); }

/* Park-Badge: versteckt, bis ot-park-on per data-lb-state gesetzt wird. */
.ot-park          { display: none; font-size: var(--fs-xs); color: var(--warn); font-weight: var
.ot-park.ot-park-on { display: inline; }

/* --- DATENGITTER ----------------------------------------------------
   opacity: var(--tile-alpha) teilt den Kachel-Transparenz-Schieberegler. */
.ot-grid {
  display: flex;
  gap: 6px;
  margin-bottom: 8px;
  opacity: var(--tile-alpha);
}
.ot-cell {
  flex: 1;
  display: flex;
  flex-direction: column;
  align-items: center;
  padding: 8px 4px;
  background: var(--card);
  border-radius: var(--radius);
}
.ot-lbl { font-size: var(--fs-xs); color: var(--label); }
.ot-val { font-size: 20px; font-weight: var(--fw-bold); color: var(--text); }
.ot-usm { font-size: var(--fs-xs); color: var(--label); }

/* --- BUTTON-REIHE ---------------------------------------------------
   opacity: var(--btn-alpha) verdrahtet den "Button-Transparenz"-Schieberegler. */
.ot-buttons { display: flex; gap: 8px; padding-bottom: 4px; }
```

```css
.ot-btn {
  flex: 1;
  display: flex;
  flex-direction: column;
  align-items: center;
  justify-content: center;
  gap: 4px;
  padding: 12px 8px;
  background: var(--btnfill);
  border-radius: var(--radius);
  cursor: pointer;
  opacity: var(--btn-alpha);
}
/* fill="currentColor" am SVG greift auf diese color-Eigenschaft zu. */
.ot-btn-icon { width: 24px; height: 24px; color: var(--bg); }
.ot-btn-lbl  { font-size: var(--fs-xs); font-weight: var(--fw-bold); color: var(--bg); }
```

---

## 15 Tipps und Einschränkungen

- **`innerHTML`-Injektion.** Dein HTML ersetzt den gesamten Inhalt von `#fold-inner` (Fold-Scope) oder `#scroller` (Full-Scope). Die App injiziert es bei jeder Theme-Aktivierung und jedem Seiten-Reload neu. Referenziere keine IDs aus dem Standard-Layout (wie `#speed-row`) - sie existieren nicht.
- **Keine externen Bilder.** `http://`- und `https://`-URLs werden aus `src`-Attributen entfernt. CSS-`background-color` und SVG-Formen für Icons und Grafiken verwenden.
- **Keine Web-Schriften aus dem Netz.** Beim System-Font-Stack bleiben oder eine base64-Font-face einbetten (Dateigröße im Auge behalten - 2-MB-Limit).
- **Standard-Scope deckt nur den Fold ab.** Dein Theme ersetzt `#fold-inner`. Nutze `"scope":"full"` in `theme.json`, um auch den Scroll-Bereich zu ersetzen (siehe Theme-Scope).
- **CSS-Ladereihenfolge.** `theme.css` wird als `active-theme.css` direkt nach `dashboard.css` im `<head>` der Seite geladen. Deine Selektoren gewinnen bei gleicher Spezifität gegen App-Regeln. Verwende eigene Klassennamen (prefixe sie), um App-UI außerhalb von `#fold-inner` nicht versehentlich zu verändern.
- **Content Security Policy.** Die Seite erzwingt `style-src file:` - nur lokale Dateien sind erlaubt. Externes CSS, Google Fonts und Icon-CDNs werden auf Browser-Ebene blockiert, noch bevor der Sanitizer überhaupt läuft. Inline-Font-face mit base64-Daten funktioniert; externe Font-URLs nicht.
- **Dunkel-/Hellmodus.** Farben nicht fest codieren - `var(--card)`, `var(--text)` usw. verwenden, damit das Theme in beiden Modi automatisch funktioniert.
- **Testen.** `telemetry.html` direkt in einem Desktop-Browser (Chrome / Firefox) öffnen. Die `LB`-Bridge fehlt, sodass das Theme-CSS und -HTML nicht aus dem Speicher geladen werden, aber du kannst: (a) dein HTML per DevTools in `#fold-inner` einfügen, (b) manuell ein `<link>` auf deine lokale `theme.css` injizieren, (c) ein `<script>` hinzufügen, das `data-lb`-Elemente mit Dummy-Werten füllt - dieses Script ist nur für Browser-Tests und muss vor dem Zippen entfernt werden.
- **Kachel-Manager-Synchronisierung.** Wenn du `data-tile-id`-Elemente hinzufügst, erscheinen ihre Labels im Theme-Manager. Halte Labels kurz (unter 20 Zeichen), damit die Liste lesbar

bleibt.

- **`rgb()`-Relative-Color-Syntax: nur flache Ursprünge.** `rgb(from var(--card) r g b / var(--tile-alpha))` funktioniert auf Android WebView 111+. `rgb(from color-mix(...) r g b / alpha)` funktioniert **nicht** - ein `color-mix()`-Ursprung verwirft den Alpha-Kanal lautlos. Als Ursprung immer eine flache CSS-Variable verwenden (z.B. `var(--card)`, `var(--value)`, `var(--btnfill)`), niemals ein `color-mix()`-Ergebnis.